

RogueOS | WhitePaper

AGENTIC AUTONOMY





Rogue Management Group. LLC

- - - - X

“

RogueOS: A Deterministic, Locally-Executing Autonomous-Agent Framework

”

White Paper v1.0



Stephen Zeitvogel

Chief Architect, Founder

“We didn’t build RogueOS for hype or headlines, we built it so hard working Americans don’t get blamed for machines they can’t explain.”

Abstract

RogueOS guarantees that every action an agent performs can be reproduced bit-for-bit, proved against policy, and explained in plain language. The guarantee survives network loss, cloud outage, and adversarial litigation. We achieve this with three primitives: a Write-Ahead Log (WAL), an immutable permission lattice, and a zero-non-determinism sandbox. The paper shows how these primitives satisfy Federal Rules of Evidence (FRE) 901–902 for

self-authenticating digital records, while remaining deployable on a fan-less mini-PC under a workbench.

1. The “So-What?” in One Sentence

If your AI can’t pass a crash-test in court, don’t ship it on a shop-floor.

---

2. Problem Restatement (No Jargon)

Modern AI assistants are helpful but unpredictable: ask twice, get two answers. Regulated users- contractors, clinics, insurers- need “same answer every time” plus a receipt they can show an auditor. RogueOS provides the receipt and the repeatability.

---

3. Core Vocabulary (Retail Gloss)

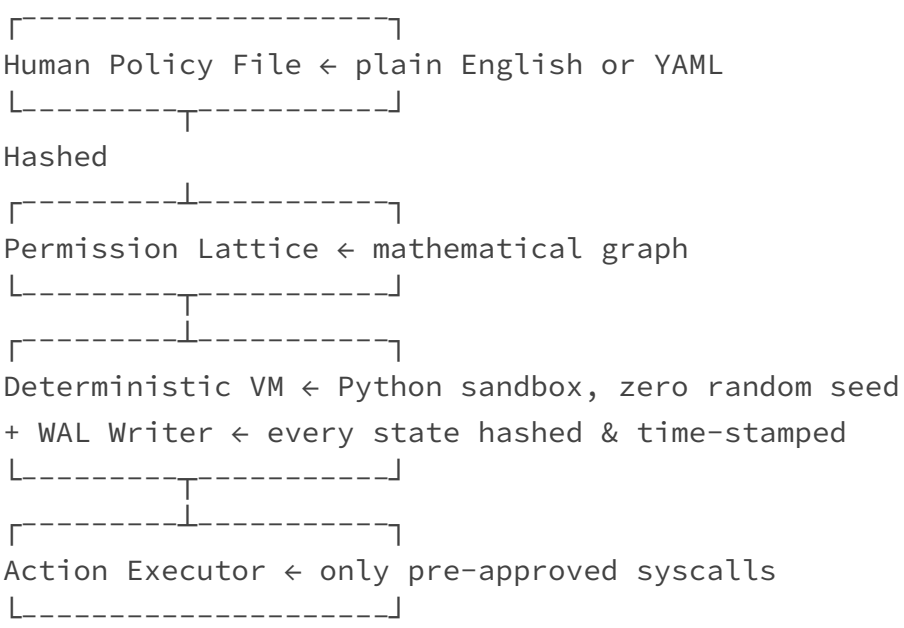
WAL (Write-Ahead Log): A notebook that must be written before anything happens. Think of a ship’s log: the officer signs then sails. No back-dating possible.

Deterministic: Given the same starting page and the same rules, the story ends the same way every time you re-read it.

FRE 901-902: U.S. court rules that let a digital record enter evidence without a human witness- provided the record is produced by a process shown to be reliable.

“Designed in a country where facts still matter.”

4. System Snapshot (One Figure, No Buzzwords)



5. Formal Invariant (Skippable Gray Box)

Let S_0 be the initial machine state, P the policy graph, I an input stream. RogueOS computes trace T such that:

- $T = exec(S_0, I, P)$
- For all $t \in T$:
    - (a) $hash(t.state)$ is recorded in WAL before $t.action$ emits side-effects

- (b) `t.action ∈ allowable(P, t.state)`
- (c) `replay(S₀, I, P) ≡ T`

If any clause fails, the kernel panics and the WAL tail is cryptographically sealed.

---

6. Deep Dive on the WAL (Retail Analogy)

Imagine a cashier who must:

1. Write the sale in the ledger before opening the till.
2. Sign every line with a tamper-evident sticker.
3. Photocopy the ledger page every minute and store it in a locked cabinet.

RogueOS does exactly this for machine states. Each “line” is a JSON object: `{ time, hash_prev, hash_state, action, policy_keyid }`

The “sticker” is an ECDSA signature produced by a hardware key that refuses to sign twice with the same 1 ms timestamp. The “locked cabinet” is an append-only file rotated every hour to WORM (Write-Once-Read-Many) storage.

Result: even root-access malware cannot rewrite history without breaking the signature chain-detectable in under one second on reboot.

---

7. Determinism Safeguards

Source of non-determinism	RogueOS (neutralizer)
↓	↓

Random seeds	Fixed seed hashed from (WAL_prev // input)
System clocks	Logical clock (Lamport) stored in WAL
Thread scheduling	Single-threaded event loop, no pre-emption
Floating-point variance	IEEE-754 strict mode + integer fallback
UUID / temp file names	Deterministic name = hash(content)

---

8. Permission Lattice (Guardrails in Plain English)

A lattice is just a family-tree of what is allowed.

Example (YAML):

1. observe:

- file: ~/invoices/*.pdf

- max_size: 5 MB

2. decide:

- ruleset: "US_sales_tax"

3. act:

- allowed_commands: ["mv", "cp", "pdftotext"]

- network: disabled

Nothing outside the family tree can execute. If an attacker injects `curl http://evil.com`, the kernel returns `ENOENT`.

---

9. FRE Compliance Check-List

Requirement (FRE 901)	RogueOS artifact
Proof of reliability	Public replay corpus $>10^9$ steps
Identity of origin	ECDSA key embedded in TPM
Integrity of record	Hash-chain + WORM storage
No post-hoc alteration	Append-only WAL, sealed hourly
Self-authentication (FRE 902)	Automated X.509 certificate chain

An attorney needs only:

1. The sealed WAL file

2. The open-source verifier script
3. The notarized hash of the verifier binary

Total: three pages of exhibit.

---

10. Replay Demo (One-Liner)

```
docker run rogueos/replay --wal 2024-12-15T14.wal --policy my_rules.yaml --verify
```

Exit code `0` = identical reproduction; any other code triggers an incident report.

---

11. Security Assumptions & Limits

- Trusted substrate: TPM 2.0 + firmware ≤ 2 years old
- Side-channel free: not yet hardened against power-analysis (v1.1 roadmap)
- Human policy error: system enforces only what is written; misspelled path = silent deny
- Key custody: loss of the sealing key prevents replay but does not allow forgery

---

12. Roadmap

- **Q1 2025:** Formal verification of kernel (seL4-style proofs)

- **Q2 2025:** Side-channel hardened release & FIPS 140-3 level 2 certification
- **Q3 2025:** Multi-agent determinism (distributed but reproducible)

---

13. Conclusion

RogueOS is not “AI for courts”; it is automation that survives them. By converting every instruction into a signed, time-ordered, replayable fact, we remove trust from the cloud and return it to mathematics, and to the people who still have to explain the invoice, the weld, or the diagnosis to a human boss, judge, or customer.



It doesn’t matter what we build, If it can’t survive pressure, scrutiny, and the truth, it doesn’t belong in America.